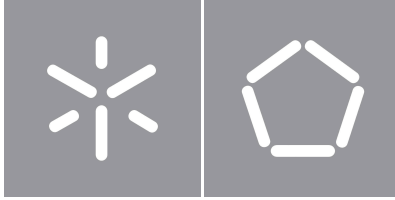


Universidade do Minho
Escola de Engenharia

Pedro Duarte Nobre Viana

Transações distribuídas em MCP



Universidade do Minho
Escola de Engenharia

Pedro Duarte Nobre Viana

Transações distribuídas em MCP

Dissertação de Mestrado em Engenharia Informática

Trabalho efetuado sob a orientação de
José Orlando Roque Nascimento Pereira

Direitos de Autor e Condições de Utilização de Trabalho de Terceiros

Esta dissertação reporta trabalho académico que pode ser utilizado por terceiros, desde que sejam respeitadas as normas e boas práticas internacionalmente aceites no que respeita aos direitos de autor e direitos conexos.

O presente trabalho pode, subsequentemente, ser utilizado nos termos estabelecidos na licença indicada abaixo.

Os leitores que necessitem de condições de autorização não previstas na licença referida deverão contactar o autor através do RepositóriUM da Universidade do Minho.

License granted to users of this work:



CC BY-NC

<https://creativecommons.org/licenses/by-nc/4.0/>

Declaração de Integridade

Declaro, por minha honra, que o presente trabalho académico foi realizado com integridade.

Confirmo que, ao longo do processo conducente à sua elaboração, não recorri a plágio, nem a qualquer forma de utilização indevida de informação ou de falsificação de resultados.

Declaro ainda que tomei pleno conhecimento do Código de Conduta Ética da Universidade do Minho.

Universidade do Minho, Braga, Fevereiro 2025

Pedro Duarte Nobre Viana

Índice

1	Introduction	1
1.1	Contexto e Motivação	1
1.2	O problema	2
1.3	Objetivos	4
2	Estado de Arte	6
2.1	Agentes LLM e Evolução para Sistemas baseados em Agentes	6
2.2	O MCP: Arquitetura e Especificação	8
2.3	Fundamentos de Transações Distribuídas	11
2.3.1	Propriedades ACID e Modelos de Consistência	11
2.3.2	Protocolos de Coordenação Atômica	12
2.3.3	Padrões de Compensação e Sagas	13
2.3.4	Mecanismos de Recuperação: WAL e DO-UNDO-REDO	14
2.4	Comparação e Seleção de Modelos	15
3	Proposta de Solução e Metodologia	17
3.1	Arquitetura do Middleware	17
3.1.1	Visão Geral e Posicionamento	17
3.1.2	Orquestrador de Transações Sagas	21
3.1.3	Gestor de Log Transacional (WAL)	21
3.1.4	Gestor de Recuperação	22
3.1.5	Interfaces e Integração com Protocolo MCP	22
3.2	Modelo Formal e Mecanismos Transacionais	22
3.2.1	Ciclo de Vida da Transação (Máquina de Estados)	22
3.2.2	Mapeamento de Operações MCP para Transações Locais	24
3.2.3	Estratégia de Compensação Automática	24
3.2.4	Protocolo de Recuperação (DO-UNDO-REDO)	24
3.3	Implementação do Protótipo	25

3.3.1	Tecnologias Utilizadas	25
3.3.2	Servidores MCP Simulados	25
3.4	Plano de Testes e Metodologia de Avaliação	26
3.4.1	Metodologia de Injeção de Falhas	26
3.4.2	Métricas de Desempenho	26
3.4.3	Comparação com Implementação Ingénua	26
3.5	Calendarização Atualizada	27
4	Conclusões e Trabalho Futuro	28
4.1	Conclusões	28
4.2	Trabalho Futuro	28
	Bibliografia	30

List of Figures

Figura 1	Arquitetura Geral do Sistema	18
Figura 2	Fluxo de Execução de uma Transação	20
Figura 3	Fluxo máquina de estados	23
Figura 4	Calendarização Atualizada	27

List of Tables

Tabela 1	Comparação técnica entre modelos de coordenação transacional	15
----------	--	----

Acronyms

2PC Two-Phase Commit.

3PC Three-Phase Commitity.

ACID Atomicity, Consistency, Isolation, Durability.

API Application Programming Interface.

HTTP Hypertext Transfer Protocol.

IA Inteligência Artificial.

IDE Integrated Development Environment.

JSON-RPC JSON Remote Procedure Call.

LLM Large Language Model.

MCP Model Context Protocol.

SDK Software Development Kit.

SSE Server-Sent Events.

TLS Transport Layer Security.

USB Universal Serial Bus.

WAL Write-Ahead Logging.

Capítulo 1

Introduction

1.1 Contexto e Motivação

A última década testemunhou uma transformação radical no panorama da IA. Os *Large Language Models* (LLMs), inicialmente concebidos como sistemas conversacionais para processamento de linguagem natural, evoluíram para agentes autônomos capazes de planejar, raciocinar e executar ações em ambientes reais [1], [2]. Esta transição, de *chatbots* que apenas respondem a perguntas para sistemas que efetivamente executam soluções, representa um dos desenvolvimentos mais significativos da IA contemporânea. Contudo, a capacidade de um agente tomar decisões inteligentes revela-se inútil se este não conseguir interagir com os sistemas, ferramentas e dados necessários para materializar essas decisões.

Até recentemente, cada integração entre um modelo de IA e um serviço externo requeria desenvolvimento *ad-hoc*: Application Programming Interface (APIs) personalizadas, lógica de autenticação específica e validação de dados. Esta fragmentação não constituía apenas um fardo de engenharia, era um claro obstáculo estrutural à padronização e à segurança. Num ecossistema onde N modelos precisavam de interagir com M ferramentas, a complexidade escalava quadraticamente ($N \times M$ integrações únicas), e cada ponto de integração representava uma potencial falha isolada sem garantias formais sobre o seu comportamento [3]. Análogamente, nos primórdios da conectividade Universal Serial Bus (USB), onde cada fabricante implementava e fabricava os seus próprios conectores e protocolos, só com a standardização surgiu a verdadeira interoperabilidade [3].

O *Model Context Protocol* (MCP), lançado pela *Anthropic* em novembro de 2024, emerge como resposta a esta fragmentação [4]. Em apenas um ano, o MCP consolidou-se como o standard para a interoperabilidade entre agentes de IA e sistemas externos, com adoção significativa: 97 milhões de downloads mensais dos *Software Development Kits* (SDK), mais de 5.800 servidores MCP publicados e validação por

gigantes tecnológicos incluindo OpenAI, Google, Microsoft e AWS [3]. Em dezembro de 2025, o protocolo foi doado à Linux Foundation sob a nova Agentic AI Foundation, assegurando uma liderança neutra e com foco numa evolução comunitária.

A arquitetura do MCP baseia-se num modelo cliente-servidor com comunicação bidirecional via *JSON Remote Procedure Call* (JSON-RPC). Um *host* (por exemplo, Claude Desktop ou uma *Integrated Development Environment* (IDE)) executa um cliente MCP que se conecta a múltiplos servidores MCP, cada um expondo recursos (bases de dados, ficheiros), ferramentas (operações invocáveis) e *prompts* (templates reutilizáveis) [3], [5]. Esta descoberta dinâmica de capacidades permite que agentes identifiquem e utilizem funcionalidades sem configuração prévia manual, disponibilizando o acesso a *workflows* complexos.

Contudo, à medida que a tecnologia transita de *proofs-of-concept* para sistemas de produção críticos e com responsabilidades, por exemplo agentes que gerem inventários, processam transações financeiras ou operam infraestruturas críticas, a tolerância ao erro diminui drasticamente. O MCP garante comunicação fiável (transporte, serialização, descoberta de capacidades) [5], mas permanece agnóstico quanto à semântica transaccional das operações que coordena. Num ambiente distribuído, onde um agente pode invocar ferramentas e manipular dados em múltiplos servidores MCP (um servidor de reservas de voos, outro de hotéis, um terceiro de pagamentos), o protocolo não oferece nenhum mecanismo para garantir atomicidade, consistência ou recuperação de falhas ou estados.

1.2 O problema

Considere-se o seguinte cenário: um agente de IA recebe a instrução de organizar uma viagem corporativa, conectando-se via MCP a três servidores distintos: um para reservas de voos, outro para alojamento hoteleiro e um terceiro para processamento de pagamentos. O agente invoca com sucesso a ferramenta `book_flight` no primeiro servidor, confirmando um voo Lisboa-Berlim com débito imediato de 450€. De seguida, ao tentar reservar o hotel através de `reserve_room`, ocorre uma falha de rede. O terceiro servidor nunca é contactado. Resultado: o utilizador fica com um voo confirmado e pago, mas sem alojamento nem transportes, um estado inconsistente que requer intervenção manual para resolver. Este não constitui um *edge case* esotérico, é a manifestação expectável do comportamento de sistemas distribuídos sem qualquer coordenação transaccional.

A arquitetura do MCP espelha exatamente um sistema distribuído clássico. Cada servidor MCP opera como um sistema autónomo, potencialmente alojado em *datacenters* distintos, implementado em *stacks* tecnológicas diferentes e gerido por organizações independentes. As operações executadas são independentes e não atómicas: o protocolo especifica como serializar pedidos JSON-RPC e como transportá-los via *stdio*, *Hypertext Transfer Protocol* (HTTP) ou SSE, mas permanece agnóstico quanto à semântica das operações invocadas [5]. Crucialmente, não existe um coordenador transaccional, nenhuma entidade que garanta que múltiplas operações em servidores diferentes constituam uma unidade lógica coesa.

O protocolo MCP na sua especificação atual, não é composto por primitivas para gestão transaccional [5]. Não existem mecanismos equivalentes a `BEGIN TRANSACTION`, `COMMIT` ou `ROLLBACK`. Não há suporte para *two-phase locking*, coordenação de *commits* distribuídos ou compensação automática de operações parcialmente executadas. O modelo de execução subjacente é *best-effort*: cada invocação de ferramenta sucede ou falha de forma isolada, sem garantias sobre o estado agregado do sistema. A responsabilidade de implementar lógica compensatória, por exemplo, cancelar o voo se a reserva de hotel falhar, recai inteiramente sobre o programador do agente, criando imensas possibilidades de erros de implementação.

As consequências desta lacuna agravam-se à medida que o MCP transita para sistemas críticos de produção. Em primeiro lugar, torna-se impossível garantir atomicidade em *workflows* distribuídos: operações como transferências bancárias entre contas ou atualizações de inventários que se espalham por múltiplos sistemas não podem ser tratadas como unidades. Em segundo lugar, surge o risco de corrupção de dados: falhas parciais podem não ser detetadas pelo agente ou pelo utilizador, resultando em estados inconsistentes que violam o domínio. Terceiro, a ausência de registo transaccional dificulta auditoria e conformidade regulamentar, sectores como a banca ou saúde exigem uma rastreabilidade completa de todas as operações, incluindo tentativas falhadas e compensações executadas. Finalmente, esta limitação constitui um bloqueio à adoção em ambientes regulados: organizações financeiras, por exemplo, não podem delegar tarefas críticas a agentes cuja fiabilidade depende de lógica compensatória *ad-hoc* sem uma validação formal.

O problema técnico em si não é novo, transações distribuídas são um domínio bastante estudado desde o trabalho de Gray nos anos 70 [6], com soluções bem estabelecidas como *Two-Phase Commit* (2PC), *Three-Phase Commit* (3PC) [7] e padrões de compensação como *Sagas* [8]. Contudo, o MCP introduz um contexto distinto que desafia a aplicação direta destas técnicas clássicas. Primeiro, o protocolo privilegia descoberta dinâmica de capacidades, isto é, os agentes não conhecem a priori que servidores estarão disponíveis ou que operações estes costumam suportar, ao contrário de sistemas transacionais

tradicionais onde os participantes são estáticos e conhecidos. Segundo, a comunicação é assíncrona e heterogénea, atravessando fronteiras de organizações e tecnologias sem controlo centralizado sobre latências ou disponibilidade. Terceiro, existe uma tensão entre a flexibilidade arquitetural do MCP, que permite servidores de curto período de vida, *stateless* ou implementados por terceiros, e os requisitos de coordenação rígida impostos por protocolos como 2PC, que assumem participantes cooperativos e capazes de manter bloqueios durante fases de preparação.

Emerge assim a questão de investigação central: Como adaptar garantias transacionais clássicas ao contexto de agentes autónomos operando sobre o MCP, preservando as propriedades: Atomicity, Consistency, Isolation, Durability (ACID), essenciais sem comprometer a flexibilidade, escalabilidade e natureza descentralizada que caracterizam o protocolo?

1.3 Objetivos

Esta dissertação visa combater a lacuna identificada através do desenvolvimento de um sistema de coordenação transacional para o MCP, preservando as propriedades ACID fundamentais sem comprometer o que caracteriza o protocolo. Este objetivo central desdobra-se em quatro secções de investigação e desenvolvimento.

Primeiro, procede-se à análise formal da arquitetura MCP e modelação transacional. Analisa-se em profundidade os fluxos de comunicação JSON-RPC entre *hosts*, clientes e servidores MCP, identificando os pontos de coordenação críticos onde garantias transacionais são necessárias. Avaliam-se padrões clássicos de transações distribuídas como o *Two-Phase Commit* (2PC), *Three-Phase Commit* (3PC) e Sagas, quanto à sua aplicabilidade neste contexto específico de agentes autónomos. Esta fase resulta na definição de um modelo que especifica a semântica transacional sobre operações MCP, incluindo condições de atomicidade e estados de transação.

Segundo, desenha-se a arquitetura de um *middleware* que servirá de coordenador transacional. Propõe-se um componente intermediário que se posiciona entre o cliente MCP e os servidores participantes, e que será responsável por orquestrar fluxos distribuídos. Este *middleware* implementa um gestor de transações que deve: (i) registar o estado de execução de *workflows* distribuídos num *log* transacional persistente; (ii) coordenar fases de preparação e confirmação entre servidores; (iii) executar compensações automáticas em caso de falha parcial; e (iv) oferecer APIs de controlo explícito (*begin*, *commit*, *rollback*) mantendo

compatibilidade com implementações MCP existentes. A arquitetura privilegia modularidade, permitindo eventuais diferentes estratégias de coordenação (2PC, Sagas) de modo a avaliar as diferentes alternativas.

Terceiro, desenvolve-se um protótipo funcional que coloca em prática a arquitetura proposta. Implementa-se o *middleware* em Python, integrando-o com o SDK oficial do MCP e validando o seu funcionamento com o uso de *mock services* que simulam servidores (base de dados, API de pagamentos, sistema de reservas). O protótipo demonstra cenários reais de uso, como o exemplo já referenciado da reserva de viagem, demonstrando recuperação automática de falhas. A implementação servirá simultaneamente como prova de conceito e como base para avaliação quantitativa.

Quarto, executa-se uma validação empírica através de injeção de falhas. Compara-se o desempenho do *middleware* transacional com uma implementação ingênua sem coordenação, avaliando duas métricas principais: (i) latência introduzida pelos protocolos de coordenação; (ii) taxa de recuperação bem sucedida perante falhas, *crashes* de servidores e *timeouts*. Ao simular diferentes perfis de falha espera-se caracterizar o comportamento do sistema em diferentes condições. Os resultados permitem equilibrar a segurança dos dados com a rapidez do sistema, podendo servir como um guia prático para a implementação desta solução em ambientes de produção.

Espera-se que este trabalho produza três contribuições principais para o estado da arte: (i) um modelo formal que estende a especificação MCP com semântica transacional, documentando extensões ao protocolo necessárias para suportar coordenação distribuída; (ii) uma implementação de referência de *middleware* transacional; e (iii) uma caracterização empírica do comportamento de diferentes estratégias transacionais neste contexto, incluindo análise de casos de uso onde cada abordagem é mais adequada. Estas contribuições têm como objetivo credibilizar e quem sabe acelerar a adoção do MCP em domínios críticos que atualmente enfrentam barreiras devido à ausência de garantias transacionais.

Capítulo 2

Estado de Arte

2.1 Agentes LLM e Evolução para Sistemas baseados em Agentes

A investigação em agentes autónomos possui uma longa tradição na comunidade académica, remontando a trabalhos em IA distribuída e robótica cognitiva. Contudo, abordagens anteriores focavam-se frequentemente no treino de agentes com conhecimento limitado em ambientes isolados, simulações controladas ou domínios restritos, divergindo significativamente dos processos de aprendizagem e adaptação observados em humanos [1]. Esta limitação tornava os agentes clássicos pouco adequados para cenários do mundo real, onde a tomada de decisão exige raciocínio sobre um outro nível de conhecimento, capacidade de generalização quando se depara com contextos imprevistos e adaptação a ambientes dinâmicos.

A emergência dos LLMs como GPT-4, Claude e Gemini alterou radicalmente este panorama. Através da aquisição de quantidades enormes de conhecimento durante o pré-treino, os LLMs demonstraram potencial para inteligência próxima do nível humano em compreensão linguística, raciocínio complexo e resolução de problemas [1], [5]. Esta capacidade impulsionou uma mudança de paradigma: de sistemas apenas conversacionais estáticos que processam interações isoladas baseada em turnos (*chatbots* puros), para agentes autónomos baseados em LLMs, sistemas que integram modelos de linguagem como controladores fundamentais em fluxos complexos, mantendo um estado partilhado entre múltiplas execuções e executando ações concretas em ambientes dinâmicos. Ao contrário de um *chatbot* que apenas responde a perguntas, um agente LLM planeia sequências de ações, invoca ferramentas externas, mantém memória de interações passadas e adapta o seu comportamento com base em *feedback* do ambiente onde opera.

Wang [1] propõem uma *framework* unificada que organiza a arquitetura de agentes LLM em quatro módulos essenciais, sintetizando a maioria do trabalho anterior nesta área. O módulo de perfil define a identidade, papel e objetivos do agente (por exemplo: assistente financeiro especializado num ambiente

bancário). O módulo de memória implementa mecanismos inspirados em capacidades cognitivas, distinguindo entre memória de curto prazo e memória de longo prazo (armazenamento vetorial externo que permite consulta e recuperação rápida de experiências passadas). O módulo de planeamento decompõe tarefas complexas em sub-tarefas executáveis através de técnicas como *chain-of-thought reasoning* ou decomposição hierárquica, reagindo a falhas com replaneamento dinâmico. Finalmente, o módulo de ação que serve de elo de ligação entre a interação entre o agente e o ambiente externo: é através deste módulo que o agente invoca ferramentas, chama APIs, manipula bases de dados ou interage com interfaces de utilizador. Este último módulo constitui o ponto de contacto crítico onde decisões do agente se transformam em operações concretas sobre sistemas reais.

A capacidade de uso de ferramentas externas (*tool use*) surge como uma característica diferenciadora dos agentes modernos. Yehudai [5] sublinha que agentes baseados em LLMs “utilizam ferramentas externas para realizar computações, aceder a conhecimento externo e interagir com o ambiente”, capacidades que estão para lá das limitações dos modelos de linguagem puros. Por exemplo, um LLM isolado não pode consultar uma base de dados em tempo real, processar uma transação financeira ou enviar um email e invocar ferramentas externas para executar essas ações. A comunidade científica desenvolveu múltiplos benchmarks para avaliar esta capacidade como o ToolBench que testa a invocação quantidades grandes de APIs reais ou APIBench avalia a seleção correta de *endpoints* em catálogos extensos [5].

A transição de LLMs de conversa para agentes autónomos implica operação em fluxos de vários passos. Ao contrário de um modelo estático que processa cada pedido independentemente, um agente integra-se num ciclo [percepção, raciocínio, ação] onde múltiplas invocações do LLM partilham o mesmo contexto e têm em consideração sobre decisões anteriores [5]. Por exemplo, reservar uma viagem corporativa pode envolver: (i) consultar disponibilidade de voos em múltiplas companhias; (ii) selecionar a opção mais adequada com base em restrições de custo e tempo; (iii) reservar o voo; (iv) identificar hotéis próximos do destino; (v) reservar alojamento; (vi) processar pagamento agregado. Cada passo depende do estado acumulado de passos anteriores e a execução sequencial atravessa múltiplos sistemas independentes (APIs de companhias aéreas, plataformas hoteleiras, métodos de pagamento). Esta natureza *stateful* e distribuída dos *workflows* de agentes modernos introduz vulnerabilidades: falhas parciais, inconsistências de estado e condições de corrida tornam-se possibilidades realistas quando as operações distribuem-se por múltiplos serviços autónomos sem coordenação explícita.

Antes de aparecerem de protocolos standardizados, cada integração entre um agente e um serviço externo requeria desenvolvimento manual: esquemas de autenticação, lógica de *serialization/deserialization*

própria e tratamento de erros *ad-hoc*. Esta divisão não só escalava a complexidade com o aumento número de agentes e ferramentas disponíveis, mas também criava barreiras à adoção em larga escala e dificultava eventuais auditorias de segurança @?. O MCP surge precisamente para resolver esta fragmentação, oferecendo uma interface standard para comunicação entre agentes (*hosts*/clientes) e serviços (MCP *servers*). Contudo, enquanto o MCP standardiza com sucesso o canal de comunicação (transporte, descoberta dinâmica de capacidades, serialização uniforme), o protocolo permanece agnóstico quanto à semântica transacional das operações que coordena. Num *workflow* distribuído onde um agente invoca ferramentas em múltiplos servidores MCP sequencialmente, não existem garantias nativas sobre atomicidade, consistência ou recuperação de estados em caso de falhas. Esta lacuna constitui o foco central desta investigação, tema aprofundado nas secções seguintes.

2.2 O MCP: Arquitetura e Especificação

O MCP responde à fragmentação de integrações identificada na secção anterior, oferecendo um standard para a comunicação entre agentes de IA e serviços externos. Proposto pela *Anthropic* em novembro de 2024 e subsequentemente doado à *Linux Foundation* (comunitário), o MCP baseia-se numa arquitetura cliente-servidor que separa responsabilidades entre três componentes fundamentais [3]. O *host* constitui a aplicação com IA disponível, por exemplo *Claude Desktop*, uma IDE ou um *chatbot* de uma corporação. É ele que orquestra a lógica de alto nível e pode conectar-se simultaneamente a múltiplos clientes MCP. O cliente MCP reside dentro do *host* e atua como intermediário, gere sessões, converte pedidos em mensagens JSON-RPC estruturadas e mantém uma relação de um para um com cada servidor. Finalmente, o servidor MCP expõe capacidades externas, ferramentas, recursos contextuais (bases de dados ou ficheiros) e *templates* reutilizáveis, funciona como sistema autónomo que pode executar localmente via *stdio* ou remotamente através de HTTP. Esta separação de responsabilidades permite que um único *host* coordene interações com dezenas de servidores diferentes sem implementar lógica específica para cada um, reduzindo a complexidade de desenvolvimento [3].

A comunicação do MCP assenta em JSON-RPC 2.0, um protocolo leve e independente da camada de transporte que define estruturas padronizadas para pedidos (*requests*), respostas (*responses*) e notificações (*notifications*). Um pedido inclui obrigatoriamente os campos `jsonrpc: "2.0"`, um identificador único `id` (*string* ou número, jamais nulo) e o `method` a invocar, opcionalmente acompanhado por `params` que guardam argumentos. Respostas referenciam o mesmo `id` do pedido correspondente e incluem um `result` em caso de sucesso ou `error` (com código, mensagem e dados opcionais) em caso de falha.

Notificações, por contraste, não precisam do campo *id*, significando que não espera nenhuma resposta, são mensagens unidirecionais usadas para atualizações em tempo real. A escolha de JSON-RPC, frequentemente descrita como “pragmática em vez de elegante”, privilegia confiabilidade e interoperabilidade sobre sofisticação arquitetural: um cliente Python pode invocar um servidor implementado em Rust, Go ou JavaScript, e o protocolo funciona indistintamente sobre *stdin/stdout*, HTTP ou *WebSockets* sem alterar a semântica das mensagens. Esta homogeneidade facilita integração com a infraestrutura existente, como *load balancers*, *API gateways* ou sistemas de autenticação, reduzindo a complexidade de adoção em ambientes já existentes.

O MCP suporta três mecanismos primários de transporte, cada um otimizado para diferentes contextos de *deployment*. O transporte *stdio* (Standard Input/Output) é predominante em integrações locais: o cliente lança o servidor como sub processo e comunica via *stdin/stdout*, criando uma conexão onde o processo servidor persiste enquanto a sessão está ativa. Este modelo é simples de implementar e beneficia de segurança herdada das permissões do sistema operativo, sendo ideal para ferramentas que acedem ao sistema de ficheiros local ou que executam operações administrativas. O transporte HTTP com *Server-Sent Events* (SSE) permite servidores remotos acessíveis via rede. O cliente envia pedidos JSON-RPC através de HTTP POST, enquanto o servidor responde via SSE *stream*, suportando notificações assíncronas e atualizações incrementais. Este transporte requer Transport Layer Security (TLS) (HTTPS) para proteger dados em trânsito. Já os *WebSockets* oferecem uma comunicação bidirecional *full-duplex* com latência inferior, mas à custa de maior complexidade de implementação. A escolha do transporte constitui uma decisão arquitetural que equilibra requisitos como latência, segurança e compatibilidade com infraestrutura existente.

MCP é um protocolo explicitamente *stateful* que impõe uma gestão rigorosa do ciclo de vida das conexões. A primeira mensagem em qualquer sessão é obrigatoriamente um pedido *initialize* do cliente ao servidor, especificando a versão do protocolo suportada (*protocolVersion*), as capacidades do cliente e outros *metadados*. O servidor responde declarando as suas próprias capacidades, que primitivas suporta (ferramentas, recursos, *prompts*) e se permite subscrições ou notificações de alterações, estabelecendo um contrato explícito sobre quais as funcionalidades disponíveis durante a sessão [4]. Este *handshake* inicial constitui uma negociação de capacidades que previne invocações de operações não suportadas. Após estabelecida, a sessão persiste através de múltiplas trocas de mensagens até um eventual encerramento explícito através de lógica de *shutdown* específica da implementação. ou implícito (*shutdown* da camada de transporte). Implementações devem estabelecer *timeouts* para *requests* enviados e, quando expiram, emitir uma notificação *cancelRequest* identificando o *request* a cancelar, eliminando a espera de uma

resposta [4]. Esta gestão de ciclo de vida *stateful* baseia-se na necessidade de manter contexto entre invocações sucessivas.

As primitivas MCP como *tools*, *resources* e *prompts*, constituem o vocabulário através do qual clientes e servidores interagem. Ferramentas representam operações invocáveis que o servidor expõe, definidas por esquemas JSON que especificam parâmetros de entrada e formatos de saída esperados, exemplos incluem *query_database*, *send_email* ou *book_flight*. Clientes invocam ferramentas através do método *tools/call*, fornecendo o nome da ferramenta e argumentos estruturados. O servidor executa a operação e retorna um resultado ou erro. Recursos encapsulam dados contextuais como bases de dados, ficheiros ou conteúdo de APIs, que os clientes podem ler via *resources/read*, recebendo *metadados* (URI, tipo MIME) e conteúdo (textual ou binário). O MCP suporta subscrições a recursos, isto é, os clientes podem registar interesse em alterações (*resources/subscribe*) e receber notificações assíncronas quando o conteúdo muda, mantendo sincronização em tempo real. *Prompts* constituem *templates* pré-definidos que *standardizam* interações comuns, por exemplo, “Resumir documento técnico” ou “Gerar relatório trimestral” isto aumenta a consistência e reduz a latência ao evitar construção manual repetitiva de instruções complexas [4]. A descoberta dinâmica destas primitivas, pois os clientes consultam servidores em tempo de execução via *tools/list*, *resources/list*, *prompts/list*, permite que os agentes adaptem o comportamento às capacidades disponíveis, sem necessidade configuração estática prévia.

Contudo, a arquitetura do MCP enfrenta uma tensão fundamental entre a necessidade de escalabilidade e o seu modelo de sessão *stateful*. Atualmente, a persistência de conexões obriga a um roteamento de rede rígido (*sticky sessions*), dificultando a escalabilidade e o uso de infraestruturas *serverless* que favorecem processos efêmeros. Com isto em mente, a especificação tem evoluído no sentido de permitir modelos de operação cada vez mais voltados para o *stateless* [4], [9]. A responsabilidade de manter o estado migra da camada de transporte para a camada de aplicação. Embora resolva desafios de infraestrutura esta mudança agrava criticamente a lacuna de coordenação, pois num ambiente *stateless* os servidores perdem a capacidade de manter contextos transacionais em memória, tornando imperativa a existência de um componente externo: um *middleware* de coordenação que garanta a atomicidade em *workflows* que atravessam múltiplos sistemas.

Por fim, a análise da especificação mostra que a robustez do canal de comunicação não se traduz em garantias sobre a consistência do estado final. O MCP assegura que uma mensagem JSON-RPC chega ao destino e é processada corretamente, mas permanece agnóstico quanto ao resultado global da operação. Num cenário onde um agente coordena ações em três servidores distintos, o protocolo não

oferece primitivas para declarar estas operações como uma unidade atômica. Não existe um mecanismo de *rollback* nem garantias de que falhas parciais sejam revertidas automaticamente. Esta lacuna num ecossistema que se move para a produção em larga escala, constitui o ponto de partida para a análise de protocolos de transação distribuída que será abordada na secção seguinte.

2.3 Fundamentos de Transações Distribuídas

2.3.1 Propriedades ACID e Modelos de Consistência

Gerir estados em sistemas distribuídos exige a definição rigorosa de garantias que assegurem a integridade dos dados, mesmo perante falhas parciais ou acessos concorrentes. Este rigor é tradicionalmente encapsulado pelo acrónimo ACID (*Atomicity, Consistency, Isolation, Durability*), um conjunto de propriedades formalizado para garantir que as transações de bases de dados são processadas de forma fiável [6].

No modelo ACID, a Atomicidade representa a indivisibilidade da operação: ou a transação é executada na sua totalidade, ou nenhum dos seus efeitos é aplicado. A Consistência assegura que uma transação apenas transita o sistema de um estado válido para outro, preservando toda a integridade do domínio. O Isolamento garante que a execução simultânea de múltiplas transações produza o mesmo estado que seria obtido se estas fossem executadas sequencialmente. Por fim, a Durabilidade assegura que, uma vez confirmado o *commit*, os efeitos da transação persistem mesmo em caso de falha ou *shutdown* do sistema [6].

No entanto, a aplicação estrita destas propriedades em ambientes distribuídos de larga escala enfrenta barreiras físicas e lógicas, sintetizadas pelo Teorema CAP de Eric Brewer [10]. O teorema mostra que é impossível para um sistema distribuído garantir simultaneamente a Consistência (todos os nós vêm os mesmos dados ao mesmo tempo), a Disponibilidade (cada pedido recebe uma resposta de sucesso ou falha) e a Tolerância a Partições (o sistema continua a operar mesmo com falhas de comunicação entre nós). Num ambiente de rede real, a tolerância a partições é uma premissa obrigatória, forçando os arquitetos de sistemas a optar entre uma consistência forte (sistemas CP) ou uma disponibilidade contínua (sistemas AP).

Esta dualidade deu origem ao modelo BASE (*Basically Available, Soft state, Eventual consistency*), que privilegia a escalabilidade e a resiliência em contrapartida da ausência de consistência imediata. Ao contrário do ACID, o modelo BASE admite que o sistema possa atravessar estados de inconsistência

temporária (*soft state*), garantindo apenas que na ausência de novos inputs todos os nós acabarão por convergir para o mesmo valor (*Eventual consistency*) [11].

A relevância desta distinção para o MCP é muito importante. Enquanto operações financeiras ou de gestão que são invocadas via servidores MCP podem exigir garantias ACID, a natureza heterogênea e descentralizada do protocolo, onde o *host* não tem controlo direto sobre a implementação dos servidores, aproxima muitos *workflows* de agentes do modelo BASE. Compreender este espectro de consistência é fundamental para determinar quando é mesmo necessário implementar mecanismos de coordenação elevada ou quando padrões de compensação assíncrona chegam.

2.3.2 Protocolos de Coordenação Atômica

A concretização das propriedades ACID, particularmente a atomicidade em ambientes distribuídos, requer protocolos de consenso que assegurem que todos os nós participantes concordam com o desfecho de uma transação. O desafio fundamental reside na incerteza da rede e na possibilidade de falhas independentes dos nós, o que exige um mecanismo de coordenação para evitar estados de inconsistência.

O 2PC [6], permanece como o padrão de referência para a coordenação atômica. O protocolo opera através de um coordenador e múltiplos participantes, estruturando-se em duas etapas distintas:

- Fase de Votação (*Prepare Phase*): O coordenador envia uma mensagem de preparação a todos os participantes. Cada participante executa localmente a transação, adquire os bloqueios (*locks*) necessários e responde com um voto de “*Commit*” (se estiver pronto) ou “*Abort*” (se algo falhar).
- Fase de Decisão (*Commit Phase*): Se o coordenador receber votos favoráveis de todos os participantes, envia uma ordem global de *Commit*. Caso isto não ocorra ou se ocorrer um *timeout*, envia uma ordem de *Abort*, indicando todos os participantes a reverterem as operações locais.

Apesar da sua eficácia, o 2PC acaba por ser bloqueante. Se o coordenador falhar após a fase de votação mas antes de transmitir a decisão, os participantes que votaram “*Commit*” ficam num estado de incerteza, mantendo recursos bloqueados para preservar a atomicidade. Esta limitação é gravosa particularmente no contexto de agentes MCP, pois a latência ou a interrupção do *host* (coordenador) pode paralisar servidores externos.

Como resposta à natureza bloqueante do 2PC, Dale Skeen propôs o 3PC [7]. Este protocolo introduz uma fase intermédia, o *Pre-Commit*, que funciona como uma barreira de segurança. Ao separar a preparação da decisão final, o 3PC permite que em caso de falha do coordenador os participantes possam chegar a uma decisão evitando o bloqueio de recursos.

Contudo, a complexidade adicional do 3PC acarreta um custo significativo em termos de latência e quantidade de mensagens trocadas, tornando-o raramente implementado em sistemas de larga escala onde a performance é extremamente importante. Para o desenvolvimento de um *middleware* para MCP, a escolha entre a simplicidade do 2PC e a resiliência do 3PC (ou alternativas como o padrão Saga, discutido a seguir) constitui uma decisão arquitetural importante.

2.3.3 Padrões de Compensação e Sagas

Conforme os sistemas evoluíram para arquiteturas de microserviços e computação em nuvem, as limitações dos protocolos atômicos (como o 2PC) tornaram-se evidentes. Nos cenários de transações de longa duração (LLTs), manter recursos que são distribuídos bloqueados durante vários segundos ou minutos prejudica gravemente a disponibilidade e a escalabilidade do sistema. O padrão Saga, introduzido por Hector Garcia-Molina e Kenneth Salem [8], surge como uma alternativa para gerir a consistência sem perder performance.

Uma Saga é definida como uma sequência de transações locais ($T_1, T_2, \dots, T_n$). Cada transação local altera o estado de um serviço autônomo e publica um evento ou mensagem que desencadeia a transação seguinte. Ao contrário do modelo ACID uma Saga não garante o isolamento. Os efeitos de T_1 tornam-se visíveis para outros processos antes de T_n ser concluída isto segue o modelo de Consistência Eventual, prescindindo do isolamento total para favorecer a disponibilidade do sistema [11]

O mecanismo principal de fiabilidade de uma Saga reside na Transação de Compensação (C_i). Para cada transação T_i , deve existir uma operação logicamente inversa C_i , capaz de reverter os seus efeitos. Se a transação T_k falhar o sistema é responsável por executar as devidas compensações ($C_{k-1}, \dots, C_1$) por ordem inversa, devolvendo o sistema a um estado anterior consistente, ainda que não necessariamente ao estado original.

Existem dois modelos principais de coordenação através Sagas:

- Coreografia: Cada serviço envolvido ouve eventos e decide qual a ação seguinte. É bastante descentralizado, mas difícil de monitorizar em ambientes complexos.
- Orquestração: Um componente centralizado (orquestrador) coordena a lógica de execução e decide quando deve invocar transações locais ou compensações.

Para o desenvolvimento de um *middleware* para o MCP, o modelo de Orquestração revela-se mais promissor. Embora o MCP estabeleça ligações persistentes entre *host* e servidor, a execução de ferramentas tende a ser atomicamente independente (*stateless*) na perspetiva do servidor. Dado que o *host* (ou um

agente intermédio) possui a visão global do plano de ação, o *middleware* pode atuar como o orquestrador que gere o *log* de estados e dispara as compensações necessárias em caso de falha de uma ferramenta (*tool call*). Esta abordagem permite que operações complexas como a reserva de múltiplos recursos, consigam ser resilientes a falhas de rede ou erros de lógica de negócio sem exigir que os servidores MCP suportem primitivas complexas de bloqueio.

2.3.4 Mecanismos de Recuperação: WAL e DO-UNDO-REDO

A fiabilidade de um *middleware* transaccional não depende apenas da correção dos seus algoritmos de coordenação, mas também da sua capacidade em restaurar um estado consistente após falhas do sistema ou interrupções. Para que um coordenador MCP possa garantir a durabilidade das operações, deve implementar um sistema de recuperação capaz de distinguir entre operações concluídas, pendentes e falhadas. Esta capacidade baseia-se em dois pilares fundamentais da teoria de bases de dados: o protocolo *Write-Ahead Logging* (WAL) e o paradigma *DO-UNDO-REDO*.

O WAL, formalizado por Gray [6], estabelece que nenhuma alteração ao estado do sistema pode ocorrer sem que a intenção dessa operação seja primeiro registada num armazenamento persistente e não volátil, ou seja, um *log*. No contexto de um *middleware* para MCP, isto implica que antes de o coordenador lançar um pedido *tools/call* para um servidor, antes deve persistir a transação, os parâmetros da chamada e o estado num *log* interno.

Este registo antecipado é o que permite a aplicação do paradigma *DO-UNDO-REDO* durante o processo de recuperação (*restart*):

- DO: Representa a execução normal da operação. No *middleware*, isto corresponde ao envio efetivo do pedido ao servidor MCP e à receção da devida resposta.
- UNDO: Caso o *middleware* detete uma falha (seja por erro do servidor ou por outro cancelamento), utiliza a informação de “UNDO” que está guardada no *log* para reverter os efeitos. No modelo de Sagas abordado na secção anterior, este mecanismo dispara a transação de compensação correspondente.
- REDO: Se o sistema sofrer um *crash* após o envio de um pedido que foi bem sucedido, mas antes do resultado ser comunicado ao agente, o mecanismo de REDO permite que o *middleware* volte a processar o *log* no arranque, conseguindo verificar o estado final da operação e garantindo que o sistema volta a atingir o estado de *committed* esperado.

A implementação destes mecanismos no *middleware* proposto resolve o problema da volatilidade que é uma vulnerabilidade crítica do protocolo MCP original. Enquanto o MCP atual é agnóstico à persistência,

uma vez que deixa a gestão de erros para a camada de aplicação, a introdução de um *log* transacional baseado em WAL assegura que o fluxo de trabalho do agente não se torna inconsistente em caso de falha de *hardware*. O *middleware* atua como uma memória persistente que protege a integridade do *workflow* distribuído [12]:

2.4 Comparação e Seleção de Modelos

A escolha de um modelo de gestão transacional para o MCP não pode ser baseada numa só propriedade, dada a diversidade de domínios onde os agentes de IA operam. Enquanto uma transação financeira exige o rigor do 2PC, um fluxo de reserva de viagens ou de automação de *emails* usufrui muito mais da flexibilidade das Sagas. Conforme apresentado na Tabela 1:

Critério	Two-Phase Commit (2PC)	Three-Phase Commit (3PC)	Padrão Saga
Garantia	ACID (Consistência Forte)	ACID (Consistência Forte)	BASE (Consistência Eventual)
Bloqueio	Bloqueante	Não bloqueante	Não bloqueante
Complexidade	Média	Muito Alta	Alta
Escalabilidade	Limitada	Muito Baixa	Elevada
Uso Ideal	Sistemas de ficheiros, DBs	Sistemas Críticos em LAN	Microserviços, APIs Web

Tabela 1: Comparação técnica entre modelos de coordenação transacional

Sintetizam-se as principais diferenças entre os modelos discutidos, servindo de base para selecionar o modelo arquitetural desta investigação.

Face à análise efetuada, esta dissertação propõe uma arquitetura de *middleware* híbrida e adaptável, assente em dois pilares:

1. Orquestração de Sagas como Modelo Principal: Considerando que o MCP é frequentemente utilizado para integrar APIs externas e sistemas variados que não suportam bloqueios de recursos (como HTTP/REST), o padrão Saga mostra-se como o mais adequado. A capacidade de executar compensações automáticas permite que o agente recupere de falhas sem bloquear os servidores participantes.

2. Implementação de WAL para Persistência e Durabilidade: Independentemente do protocolo de coordenação (Saga ou 2PC), o *middleware* implementará obrigatoriamente um mecanismo de WAL. Esta decisão visa mitigar a natureza *stateless* do protocolo MCP, garantindo que o coordenador possa executar o processo de REDO após uma falha de sistema, assegurando que o *workflow* do agente nunca fique num estado incerto.

Em suma, a solução desenvolvida irá privilegiar a disponibilidade e a resiliência (modelo AP do teorema CAP), oferecendo ao utilizador a segurança de que o sistema será capaz de reverter para um estado consistente através de transações de compensação devidamente orquestradas.

Capítulo 3

Proposta de Solução e Metodologia

3.1 Arquitetura do Middleware

3.1.1 Visão Geral e Posicionamento

A arquitetura proposta nesta dissertação baseia-se num *middleware* de coordenação transacional que se posiciona estrategicamente entre o cliente MCP e os servidores MCP que expõem ferramentas e recursos. Este posicionamento ilustrado na Figura 1, permite que o *middleware* intercete todas as invocações de ferramentas realizadas pelo agente, orquestrando-as segundo semântica transacional sem exigir modificações aos servidores MCP existentes nem alterações ao protocolo MCP subjacente. A preservação da compatibilidade global constitui um requisito fundamental, pois os servidores MCP desenvolvidos independentemente por terceiros que podem não ter conhecimento ou interesse em suportar coordenação transacional continuam operacionais sem qualquer adaptação, enquanto o *middleware* garante atomicidade e recuperação de falhas de forma transparente.

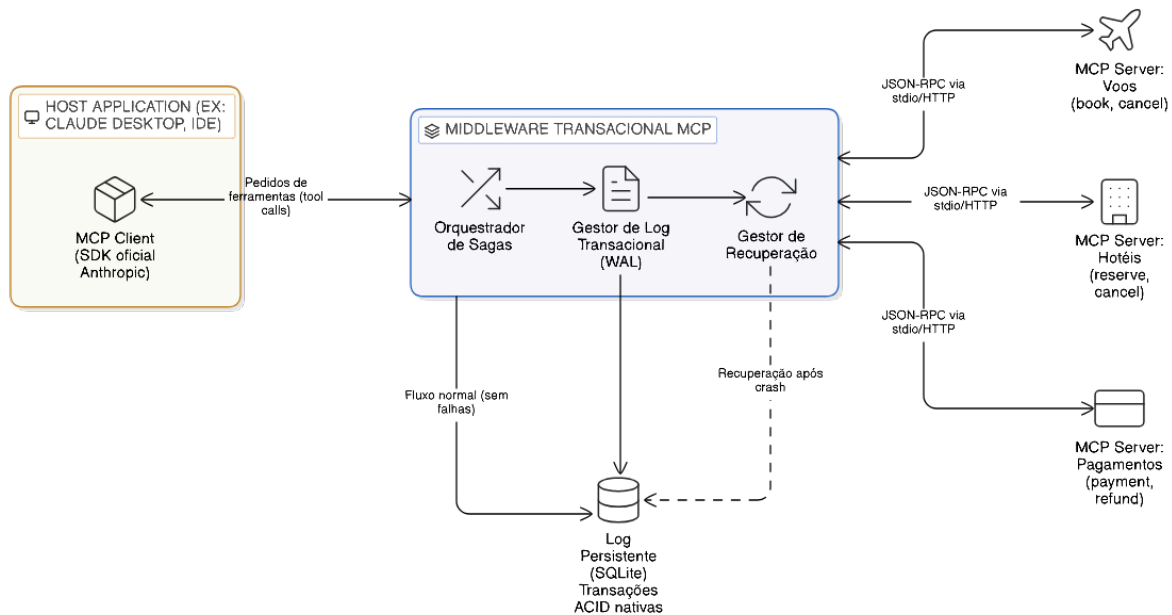


Figura 1: Arquitetura Geral do Sistema

Do ponto de vista funcional, o *middleware* comporta-se como um *proxy* que serve de mediador entre a comunicação do cliente com os servidores. Quando o agente LLM decide executar um *workflow* que envolve múltiplas operações distribuídas como por exemplo, reservar sequencialmente um voo, hotel e processar pagamento, o *host* emite uma sequência de pedidos *tools/call* através do cliente MCP. Em vez de estes pedidos serem enviados diretamente aos servidores, o *middleware* interceta-os e regista-os no *log* antes de os encaminhar. Esta abordagem de WAL garante que mesmo ocorrendo falhas catastróficas (como *shutdown* do *host* ou a perda de conexão), o estado da transação continua recuperável a partir do armazenamento persistente. O *middleware* mantém assim uma representação persistente do progresso do *workflow* e permite que transações incompletas sejam retomadas ou compensadas quando o sistema reiniciar.

A arquitetura distingue-se de abordagens alternativas, como extensões ao protocolo MCP ou modificações aos servidores individuais, porque opta por um modelo pouco intrusivo. Extensões do protocolo MCP, embora mais elegantes, exigiriam um consenso comunitário e uma adoção coordenada por parte dos utilizadores, processos que podem levar anos e podem enfrentar eventual resistência. Já modificações diretas nos servidores MCP fragmentariam o ecossistema, pois servidores que fossem transacionalmente complexos e servidores *legacy* coexistiriam, criando inconsistências de comportamento e dificultando o funcionamento do sistema. O *middleware* proposto resolve ambos os problemas porque atua como um adaptador silencioso. Do ponto de vista do *host* interagem através da interface MCP, e os servidores recebem pedidos MCP válidos indistinguíveis de invocações diretas.

Conforme ilustrado na Figura 2, o fluxo de execução de uma transação demonstra a coordenação entre os componentes do *middleware* e os servidores MCP.

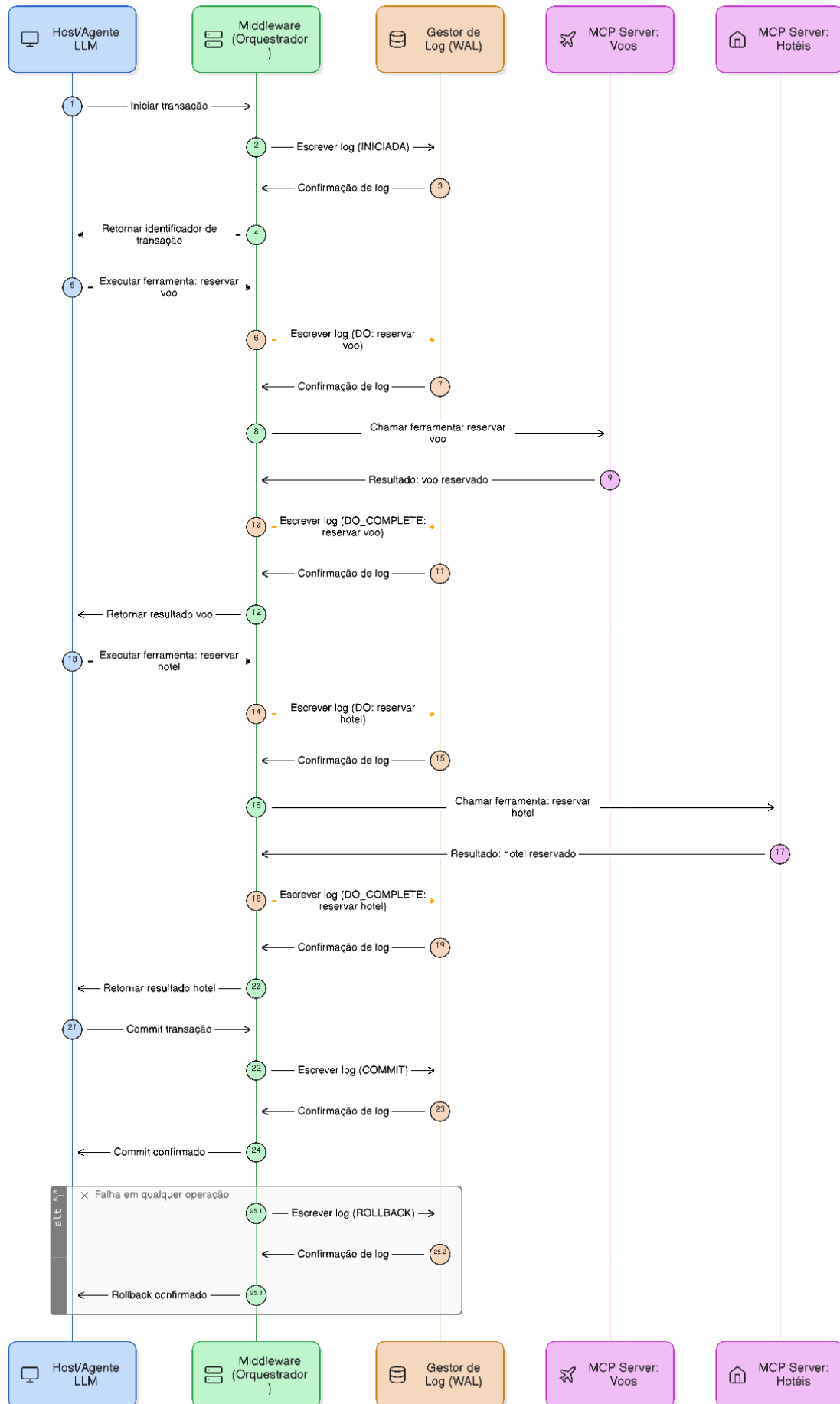


Figura 2: Fluxo de Execução de uma Transação

Em síntese, a arquitetura deste *middleware* junta o rigor dos modelos teóricos de Sagas e WAL com as restrições práticas do ecossistema MCP. Ao optar por uma solução externa e modularmente invisível, garante-se a compatibilidade com as ferramentas existentes. Nas secções que se seguem, detalha-se a implementação dos seus componentes, iniciando-se pelo orquestrador de Sagas que será o núcleo responsável pela lógica de coordenação do sistema.

3.1.2 Orquestrador de Transações Sagas

O núcleo funcional da arquitetura proposta é o Orquestrador de transações, responsável por coordenar a execução de *workflows* distribuídos segundo o padrão de Sagas. Este componente atua como a peça central que mantém uma visão global da transação distribuída, decidindo a sequência de invocações de ferramentas MCP, monitorizando o seu progresso e desencadeando mecanismos de compensação em caso de falhas.

O Orquestrador modela o ciclo de vida das transações através de uma máquina de estados que distingue entre fases de execução normal, compensação em caso de falha, e finalização (sucesso ou abortar).

O Orquestrador mantém uma representação explícita do plano de execução do *workflow*, modelando cada ação do agente como uma transação local independente. Cada transação local corresponde normalmente a uma invocação `tools/call` sobre um servidor MCP específico, por exemplo `book_flight`, `reserve_room` ou `process_payment`. Para cada uma destas ações é definida uma ação de compensação inversa, como `cancel_flight` ou `refund_payment`, de forma a permitir reverter os efeitos da operação em caso de falha.

A coordenação segue um modelo estritamente sequencial e determinístico, ou seja, o Orquestrador apenas autoriza a progressão para a transação T_{n+1} após a confirmação de sucesso de T_n . Caso ocorra uma falha o Orquestrador interrompe o fluxo direto e inicia a fase de reversão, executando as compensações das tarefas previamente concluídas. Esta abordagem retira a responsabilidade da gestão de erros da lógica do LLM, garantindo que a recuperação de falhas é tratada de forma programada e previsível.

3.1.3 Gestor de Log Transacional (WAL)

Para assegurar a propriedade de Durabilidade, a arquitetura integra um Gestor de *Log* baseado no princípio de WAL. Este módulo é o que garante a persistência do *middleware*, assegurando que nenhum pedido é encaminhado para um servidor MCP sem que a sua intenção tenha sido previamente registada em armazenamento.

O Gestor de *Log* mantém um registo sequencial (*append-only*) que armazena o contexto completo de cada operação como o identificador da Saga, os parâmetros da ferramenta invocada e o estado atual da

transação. Esta persistência é importante porque permite que o *middleware* seja agnóstico a falhas do próprio processo porque caso o *middleware* sofra um *crash* durante uma operação crítica, o *log* servirá de memória para o processo de reconstrução do estado no arranque seguinte.

3.1.4 Gestor de Recuperação

O Gestor de Recuperação é o componente ativado quando o *middleware* inicia. A sua função é analisar o estado do *log* transacional deixado pela última sessão e identificar transações que ficaram num estado indeterminado, por exemplo operações registadas como “em curso” mas sem confirmação de sucesso.

Através do paradigma `_DO-UNDO-REDO_`, o Gestor de Recuperação decide se deve tentar completar uma operação pendente ou se deve disparar o protocolo de compensação para reverter o sistema para o último estado seguro conhecido. Este módulo fecha o ciclo de resiliência da arquitetura, transformando o *middleware* num sistema capaz de sobreviver não só a falhas externas (servidores MCP), mas também a falhas internas.

3.1.5 Interfaces e Integração com Protocolo MCP

A integração com o ecossistema MCP é realizada através de uma camada de abstração que emula as interfaces padrão do protocolo. Do ponto de vista do Agente (Host), o *middleware* apresenta-se como um cliente MCP robusto; do ponto de vista dos Servidores, o *middleware* atua como um coordenador transparente. Esta secção de interface assegura que a adição de garantias transacionais não prejudica a compatibilidade com ferramentas existentes, permitindo uma adoção incremental da solução.

3.2 Modelo Formal e Mecanismos Transacionais

3.2.1 Ciclo de Vida da Transação (Máquina de Estados)

A fiabilidade do *middleware* assenta na capacidade de monitorizar e gerir o estado de cada interação entre o agente e os servidores MCP. Para evitar inconsistências cada unidade de trabalho é tratada como uma transação distribuída direcionada por uma Máquina de Estados Finitos (FSM). Este formalismo permite que o sistema determine, de forma inequívoca, qual a ação a tomar perante uma interrupção de rede ou uma falha de sistema.

O ciclo de vida de uma transação no *middleware* é composto pelos seguintes estados fundamentais:

INITIATED: O estado inicial que é ativado assim que o *middleware* interceta um pedido `tool/call`. Aqui a intenção da transação é registada no *log*, mas nenhum pedido foi enviado ao servidor destino.

EXECUTING: O pedido foi despachado para o servidor MCP. O *middleware* permanece num estado de espera ativa, aguardando a resposta JSON-RPC. Se o sistema falhar neste estado, o Gestor de Recuperação assumirá que a operação pode ter sido executada e irá escolher entre o REDO (tentar obter o resultado) ou o UNDO (compensar).

COMMITTED: Recebida a confirmação de sucesso do servidor, a transação é marcada como concluída. O *middleware* atualiza o *log* e avança no fluxo para a próxima instrução do agente.

FAILED: Este estado é atingido se o servidor MCP devolver um erro de aplicação (ex: “lugar indisponível”) ou se ocorrer um erro de transporte (ex: timeout). Este estado funciona como um gatilho para a transição para a fase de compensação.

COMPENSATING: O *middleware* inicia a execução das ferramentas de reversão associadas às transações que já tinham sido concluídas com sucesso. O sistema permanece neste estado até que todo o histórico da Saga tenha sido processado de forma inversa e seja atingido um estado estável.

ABORTED: O estado terminal após uma falha. Indica que o sistema foi revertido com sucesso e encontra-se novamente num estado consistente.

A Figura 3 ilustra a máquina de estados completa, mostrando as transições entre os diferentes estados.

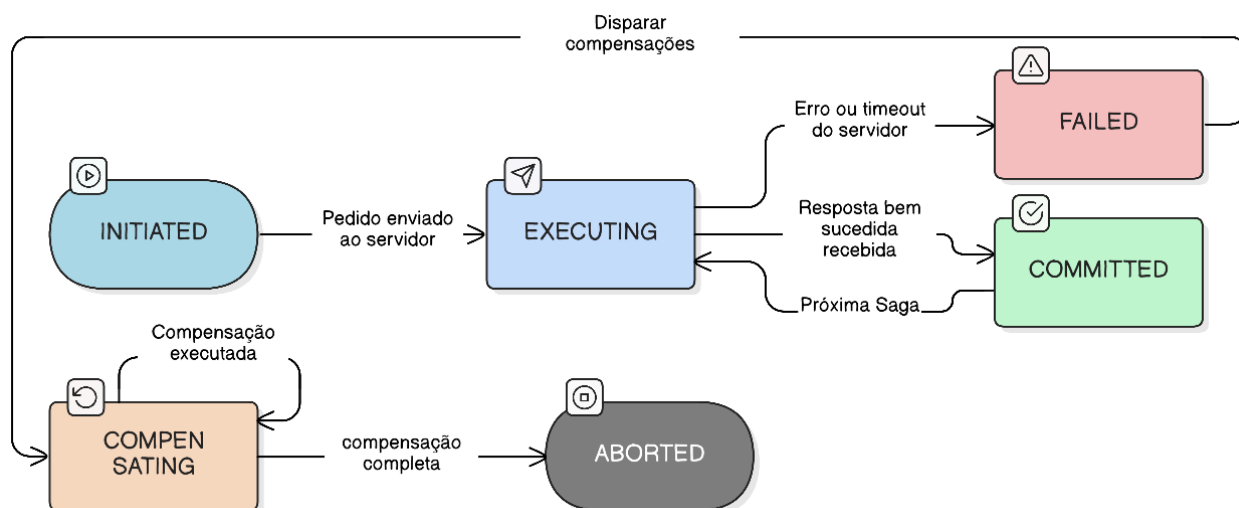


Figura 3: Fluxo máquina de estados

Este formato garante que a qualquer altura, o *middleware* possui consciência do progresso do *workflow*. Isto assegura que o tratamento de erros não é deixado ao acaso ou à interpretação do LLM.

3.2.2 Mapeamento de Operações MCP para Transações Locais

Para que o Orquestrador consiga controlar a consistência, o *middleware* precisa de uma camada de tradução que associe cada *tool call* ao seu contexto em termos transacionais. No modelo proposto cada operação T_i solicitada pelo agente é tratada como uma transação local dentro de uma Saga global.

Quando o agente emite um pedido, o *middleware* inspeciona o id da ferramenta e extrai as regras para a compensação. Caso uma ferramenta não tenha uma compensação definida o *middleware* pode optar por duas abordagens, ou emite um aviso ao agente de que a operação é irreversível ou impede a execução dentro de um bloco transacional para evitar estados irreversíveis e inconsistentes. As duas abordagens são válidas e serão consideradas de forma a atingir melhores métricas.

3.2.3 Estratégia de Compensação Automática

A estratégia de compensação adotada segue uma lógica LIFO (*Last-In, First-Out*), garantindo que a reversão do sistema ocorre na ordem inversa à execução original. Esta escolha é importante em sistemas distribuídos porque as operações posteriores podem depender do estado criado por operações anteriores.

O fluxo de compensação é disparado automaticamente quando a máquina de estados atinge o estado FAILED. O processo segue estes passos:

1. Interrupção: O fluxo direto é imediatamente suspenso.
2. Identificação: O Orquestrador consulta o *log* para identificar todas as transações daquela Saga que já atingiram o estado COMMITED.
3. Inversão: O *middleware* executa as transações de compensação ($C_n, C_{n-1}, \dots$) de forma sequencial.

3.2.4 Protocolo de Recuperação (DO-UNDO-REDO)

De forma a complementar o algoritmo de Sagas, o *middleware* implementa um protocolo de recuperação simplificado para lidar com falhas catastróficas (como o *shutdown* repentino do processo). Este protocolo utiliza a informação persistida no *log* transacional para decidir as ações no *restart* do sistema:

- REDO: Se o *log* indicar que uma operação foi concluída com sucesso no servidor MCP, mas o resultado ainda não foi comunicado ao agente, o *middleware* tenta retransmitir essa confirmação.
- UNDO: Se o *log* mostrar uma operação em estado EXECUTING sem confirmação do desfecho, o sistema assume que a atomicidade foi violada e inicia a compensação para garantir que não há efeitos parciais.

Esta abordagem garante que a propriedade da Durabilidade é mantida mesmo em condições adversas. Ao introduzir o modelo DO-UNDO-REDO, o *middleware* deixa de ser apenas um facilitador de mensagens

para se tornar um sistema tolerante a falhas, capaz de garantir que, independentemente de quando ocorra uma interrupção, o sistema consegue sempre e eventualmente convergir para um estado consistente.

3.3 Implementação do Protótipo

3.3.1 Tecnologias Utilizadas

A implementação do protótipo proposto nesta dissertação é feita em *Python*, recorrendo ao SDK oficial do MCP disponibilizado pela *Anthropic*. A escolha de *Python* justifica-se por três fatores principais. Primeiro, trata-se de uma linguagem amplamente utilizada no ecossistema de agentes baseados em LLMs, dispondo de bibliotecas para comunicação assíncrona, serialização de JSON e persistência de dados. Em segundo lugar, a linguagem facilita a prototipagem rápida e a experimentação de diferentes estratégias de coordenação transacional, objetivo central desta fase do trabalho. Finalmente, a compatibilidade direta com o SDK oficial do MCP reduz o risco de desvios face à especificação e garante que os resultados obtidos representam cenários reais de utilização do protocolo.

O SDK oficial do MCP é utilizado para implementar tanto o cliente intermediário (integrado no *middleware*) como os servidores MCP simulados [4]. Isto permite validar a arquitetura proposta sem recorrer a abstrações artificiais, garantindo que todas as interações respeitam o modelo JSON-RPC definido na especificação do protocolo. O *middleware* é implementado como um processo independente que encapsula o cliente MCP e introduz lógica adicional de coordenação, persistência e recuperação, mantendo-se transparente quer para o *host* quer para os servidores.

3.3.2 Servidores MCP Simulados

Para validar o comportamento do *middleware* em cenários controlados, são desenvolvidos servidores MCP *Mock* que representam serviços externos normalmente utilizados por agentes autónomos. Estes serviços simulados vão permitir testar a orquestração de Sagas sem dependência de sistemas reais, facilitando a injeção de falhas e a observação do comportamento do sistema.

Cada servidor MCP *Mock* é implementado de forma a permitir a simulação de falhas como atrasos, erros ou *shutdown* repentino do processo. Esta capacidade é essencial para avaliar o quão robusto é o *middleware* e verificar se os mecanismos de compensação e recuperação são corretamente acionados.

3.4 Plano de Testes e Metodologia de Avaliação

3.4.1 Metodologia de Injeção de Falhas

A avaliação experimental do *middleware* baseia-se numa metodologia de injeção controlada de falhas desenhada para refletir condições realistas de operação em sistemas distribuídos. As falhas testadas incluem:

- Falhas de rede simuladas através de *timeouts*, latência elevada e perda de mensagens entre o *middleware* e os servidores MCP.
- Falhas de servidor representadas pelo encerramento abrupto de um servidor MCP durante a execução de uma transação local.
- Falhas do *middleware* simulando *crashes* do processo de após a execução parcial de um workflow.

Após cada falha, o sistema é reiniciado e o mecanismo de recuperação é acionado, permitindo avaliar a correção das operações de compensação e de recuperação do estado.

3.4.2 Métricas de Desempenho

A avaliação do sistema baseia-se em duas métricas principais:

- Latência Introduzida: medida como o tempo adicional necessário para concluir um *workflow* distribuído quando comparado com uma execução direta via MCP sem *middleware*. Esta métrica permite quantificar o *overhead* associado à coordenação transacional e à persistência em *log*.
- Taxa de Recuperação Bem sucedida: definida como a percentagem de execuções em que o sistema consegue recuperar automaticamente para um estado consistente após uma falha. Esta métrica avalia diretamente a eficácia do modelo implementado e dos mecanismos de recuperação.

3.4.3 Comparação com Implementação Ingénua

Para demonstrar o valor do *middleware* proposto, os resultados obtidos são comparados com uma implementação *baseline* que utiliza o MCP de forma direta e ingénua, sem qualquer mecanismo de coordenação transacional ou persistência. Nesta implementação ingénua, o agente invoca sequencialmente ferramentas MCP e trata falhas de forma isolada, sem compensações automáticas.

A comparação vai permitir evidenciar diferenças claras entre as duas abordagens principalmente no que diz respeito à ocorrência de estados inconsistentes e à necessidade de intervenção manual após

falhas. Esta análise é fundamental para justificar a complexidade adicional introduzida pelo *middleware* e demonstrar a sua aplicabilidade real e prática.

3.5 Calendarização Atualizada

A execução desta dissertação foi planeada de forma a garantir os prazos académicos do Mestrado em Engenharia Informática. O cronograma abaixo (Figura 4) reflete o estado atual do projeto.

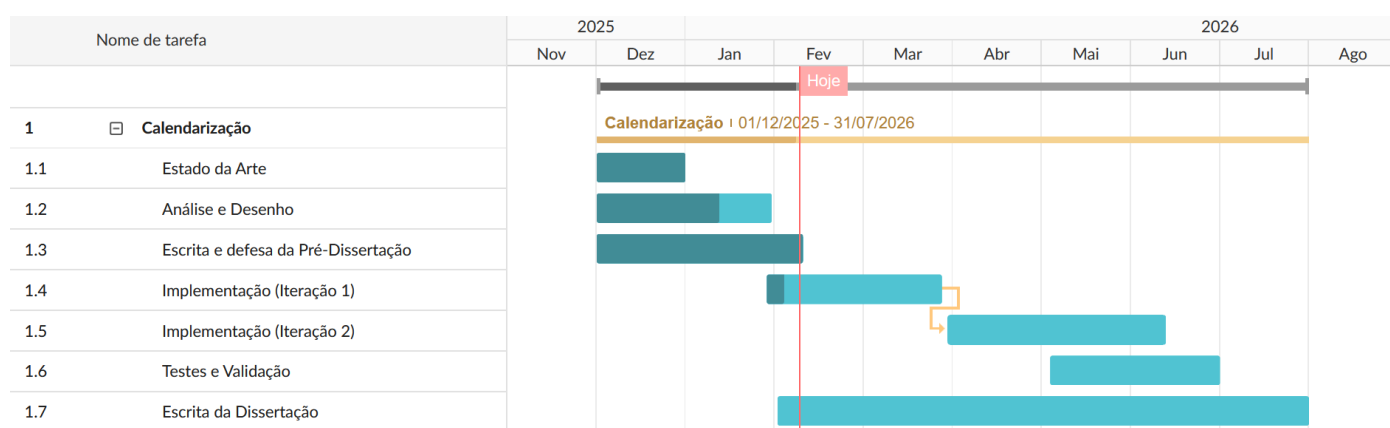


Figura 4: Calendarização Atualizada

O trabalho está estruturado nas seguintes etapas principais:

1.1 Estado da Arte: Revisão bibliográfica sobre MCP, LLM *Agents*, e Transações Distribuídas.

1.2 Análise e Desenho: Definição da arquitetura da solução. Escolha das tecnologias e desenho dos fluxos transacionais.

1.3 Escrita e defesa da Pré-Dissertação

1.4 Implementação (Iteração 1): Configuração do ambiente MCP. Criação dos *mock services* para teste.

1.5 Implementação (Iteração 2): Desenvolvimento do mecanismo de gestão de transações e lógica de compensação.

1.6 Testes e Validação: Simulação de falhas, recolha de métricas e análises comparativas.

1.7 Escrita da Dissertação: Redação da dissertação de mestrado

Atualmente, o trabalho encontra-se no início da etapa 1.4.

Capítulo 4

Conclusões e Trabalho Futuro

4.1 Conclusões

Esta pré tese apresentou o enquadramento conceptual e metodológico de um trabalho de dissertação centrado na introdução de garantias transacionais em *workflows* distribuídos baseados no MCP. A análise realizada evidenciou que apesar da crescente adoção do MCP como mecanismo para integrar agentes baseados em LLMs e serviços externos, o protocolo não fornece mecanismos de coordenação transacional.

Com base nesta lacuna foi definida uma proposta de solução assente num *middleware* transacional externo ao protocolo, inspirado no padrão de Sagas orquestradas e em técnicas clássicas de recuperação de sistemas distribuídos. A arquitetura proposta procura equilibrar robustez e compatibilidade, evitando alterações à especificação do MCP e preservando a interoperabilidade com servidores existentes.

Foi também descrito o modelo transacional subjacente, a estratégia de implementação do protótipo e a metodologia de avaliação experimental a adotar na fase seguinte do trabalho.

Este enquadramento permite estabelecer uma base sólida para o desenvolvimento e validação da solução proposta no contexto da dissertação.

4.2 Trabalho Futuro

O trabalho futuro desta dissertação consistirá inicialmente na implementação do protótipo de *middleware* transacional descrito, recorrendo a servidores MCP simulados para validação funcional e experimental.

Esta implementação vai permitir verificar a viabilidade do modelo proposto e identificar limitações de decisões de design tomadas ao nível da arquitetura.

Numa fase posterior será conduzida uma avaliação experimental, baseada na injeção controlada de falhas e na medição de métricas de desempenho previamente definidas. Esta avaliação vai permitir comparar o comportamento do sistema com abordagens básicas sem suporte transacional, e espera-se que forneça evidências sobre os *trade-offs* da solução envolvidos.

Finalmente, os resultados obtidos vão servir de base para uma análise crítica do quão adequado se mostra o padrão de Sagas no contexto do MCP, bem como para a identificação de possíveis extensões ou melhorias, tanto ao nível do *middleware* proposto como da própria evolução do protocolo. Estas etapas constituem o núcleo do trabalho a desenvolver na dissertação final.

Bibliografia

- [1] L. Wang, C. Ma, e X. Feng, «A Survey on Large Language Model based Autonomous Agents». [Em linha]. Disponível em: <https://arxiv.org/abs/2308.11432>
- [2] Anthropic, «Introducing the Model Context Protocol». [Em linha]. Disponível em: <https://www.anthropic.com/news/model-context-protocol>
- [3] D. Gupta, «The Complete Guide to Model Context Protocol (MCP): Enterprise Adoption, Market Trends, and Implementation Strategies». [Em linha]. Disponível em: <https://guptadeepak.com/the-complete-guide-to-model-context-protocol-mcp-enterprise-adoption-market-trends-and-implementation-strategies/>
- [4] Anthropic, «Model Context Protocol Specification». [Em linha]. Disponível em: <https://modelcontextprotocol.io/specification>
- [5] A. Yehudai, «Survey on Evaluation of LLM-based Agents». [Em linha]. Disponível em: <https://arxiv.org/abs/2503.16416>
- [6] J. Gray, *Notes on Data Base Operating Systems*. Springer-Verlag. [Em linha]. Disponível em: <https://jimgray.azurewebsites.net/papers/dbos.pdf>
- [7] D. Skeen, «Nonblocking commit protocols», *EFCS Department, University of California*. [Em linha]. Disponível em: <https://web.eecs.umich.edu/~manosk/assets/papers/Ske81.pdf>
- [8] H. Garcia-Molina e K. Salem, «Sagas». [Em linha]. Disponível em: <https://www.cs.cornell.edu/andru/cs711/2002fa/reading/sagas.pdf>
- [9] GitHub, «SEP-1442: Make MCP Stateless (by default)». [Em linha]. Disponível em: <https://github.com/modelcontextprotocol/modelcontextprotocol/issues/1442>
- [10] Brewer E., *Towards robust distributed systems*.

- [11] Vogels W., «Eventually Consistent», *Communications Of The ACM*. [Em linha]. Disponível em: https://www.researchgate.net/publication/27301367_Eventually_Consistent
- [12] Haerder T. e Reuter A., «Principles of Transaction-Oriented Database Recovery», *University of Kaiserslautern, West Germany*. [Em linha]. Disponível em: <https://cs-people.bu.edu/mathan/reading-groups/papers-classics/recovery.pdf>